

How Attendance Hours Are Calculated

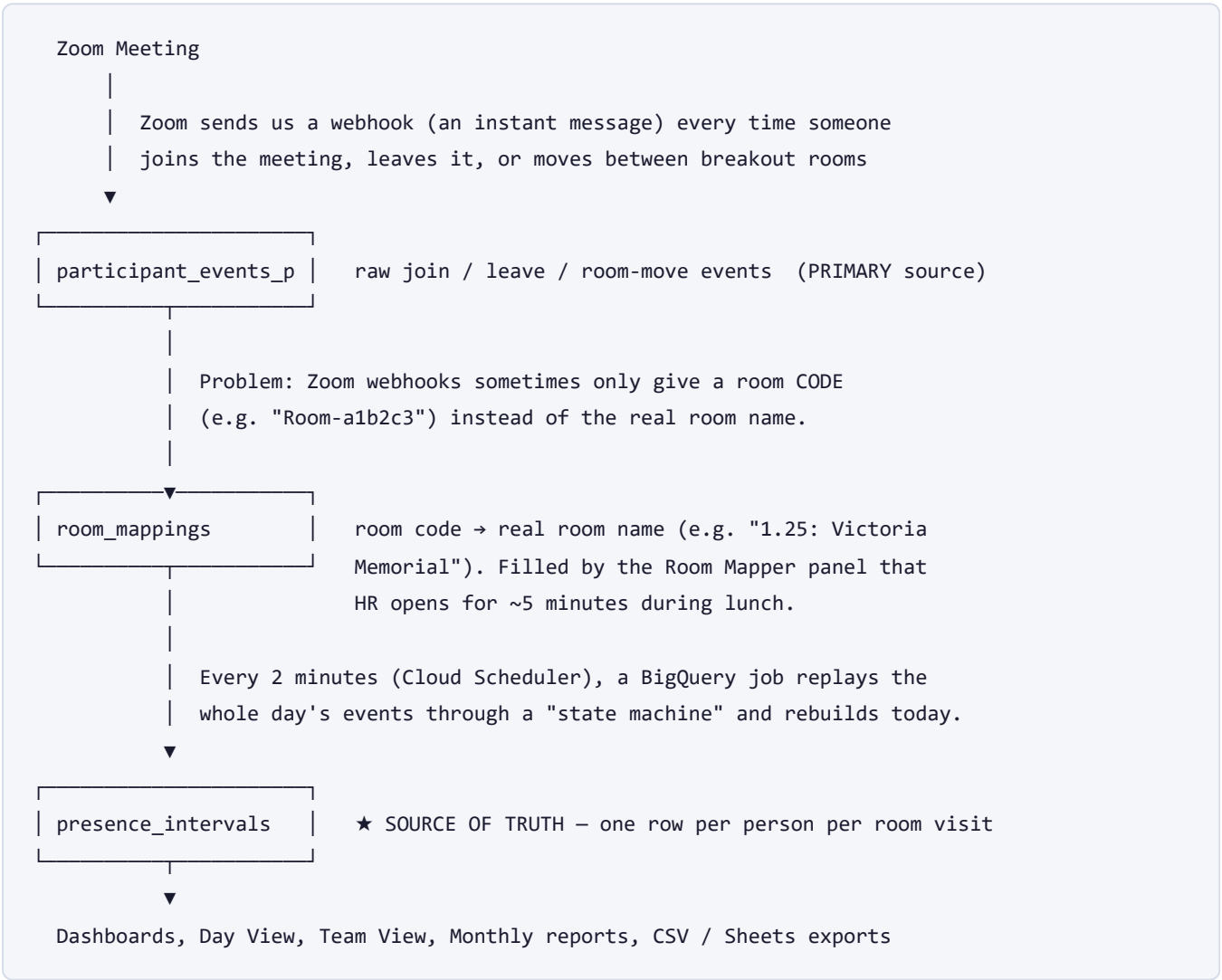
The **presence_intervals** table — data flow, rules, schema, and the query behind it • Verve Attendance Tracker • 27 July 2026

1. The Big Picture

Every report in the dashboard (Day View, Team View, Monthly, CSV exports) reads hours from **one single table**: `presence_intervals` in BigQuery. Nothing calculates hours anywhere else.

The idea is simple: for every person, for every room they sat in, we store one row that says *"this person was in this room from this time to this time."* Adding up those rows gives the person's working hours for the day.

2. Data Flow — where the data comes from



A third table, `room_snapshots_v2` (old bot data), is only used to help match a person's display name to their email. It plays no part in the hours themselves for current dates.

3. The Raw Tables — what goes in

Three tables feed the build. Only the first one contains the actual time data; the other two just help with naming and identity.

3.1 `participant_events_p` — the webhook events (PRIMARY source of all hours)

One row per Zoom webhook. Partitioned by `event_date`. This is the only table the times come from.

Field	Type	What it means / how the build uses it
<code>event_id</code>	STRING	Unique ID of the webhook event (for deduplication).
<code>event_type</code>	STRING	What happened. The build only uses 6 types: <code>participant_joined</code> / <code>participant_left</code> (and their <code>meeting.*</code> variants) to open/close presence windows, and <code>breakout_room_joined</code> / <code>breakout_room_left</code> to know which room the person moved to.
<code>event_timestamp</code>	TIMESTAMP	The exact moment it happened (UTC). Every <code>start_ts</code> / <code>end_ts</code> in <code>presence_intervals</code> comes from these timestamps.
<code>event_date</code>	DATE	IST date of the event. Partition key — the build reads two partitions (day D and D+1) to capture midnight-crossing sessions.
<code>meeting_id</code>	INTEGER	Zoom meeting number. Stored as a number, so the query does <code>CAST(meeting_id AS STRING)</code> before use.
<code>meeting_uuid</code>	STRING	Unique ID of this specific meeting instance (changes if the meeting restarts).
<code>participant_id</code>	STRING	Zoom's ID for the participant in this meeting.
<code>participant_name</code>	STRING	Display name as typed in Zoom. Cleaned (suffixes stripped, lowercased) to group a person's events together; names containing "scout" (the bot) are dropped.
<code>participant_email</code>	STRING	Email if Zoom sent one. Used to build the day's name→email identity map.
<code>room_uuid</code>	STRING	Zoom's code for the breakout room (e.g. <code>6kAkE8j0geGj5m2DPy9/</code>). Rotates every day/meeting instance. Join key to <code>room_mappings</code> .
<code>room_name</code>	STRING	Room name as delivered in the webhook. This is the authoritative room name — unless it is a placeholder like <code>Room-a1b2c3</code> (happens when the name wasn't known yet at insert time).
<code>inserted_at</code>	TIMESTAMP	When our server wrote the row.

3.2 `room_mappings` — room code → real room name

Filled by the Room Mapper panel (the ~5-minute HR session): the SDK sees who is in which named room, webhooks say which room *code* those same people are in, so code = name. One row per resolved room per day.

Field	Type	What it means / how the build uses it
<code>mapping_id</code>	STRING	Unique ID of the mapping row.

meeting_id	INTEGER	Zoom meeting number (numeric here too — same CAST rule applies).
meeting_uuid	STRING	Meeting instance the mapping belongs to.
room_uuid	STRING	The webhook room code being mapped. Join key against participant_events_p.room_uuid.
room_name	STRING	The real, human-readable room name (e.g. 1.25: Victoria Memorial).
room_index	INTEGER	Position of the room in the meeting's room list (legacy calibration ordering).
mapping_date	DATE	The day the mapping is valid for — room codes rotate daily, so mappings are only ever used for their own date .
mapped_at	TIMESTAMP	When the mapping was saved. Used for "newest knowledge wins": if the event stream later saw a real name newer than this, the event name is trusted instead.
source	STRING	How the mapping was made — e.g. webhook_primary_sdk_lookup (current Room Mapper), webhook_calibration / zoom_sdk_app / timestamp_calibration (legacy bot calibration).

3.3 room_snapshots_v2 — old bot polling data (identity helper only)

Until 20 July 2026 the Scout Bot polled the meeting every 30 seconds and wrote one row per person per room per poll. The bot is now retired: for current dates this table is **only read to learn name→email pairs** (step 2 of the query); it contributes no time. Historical dates before the cutover were built from it directly.

Field	Type	What it means
snapshot_id	STRING	Unique ID of the poll row.
snapshot_time	TIMESTAMP	When the 30-second poll ran.
event_date	DATE	IST date of the snapshot (partition key).
meeting_id	STRING	Zoom meeting number.
room_name	STRING	Room the person was seen in (SDK gives real names directly).
participant_name	STRING	Display name seen by the SDK.
participant_email	STRING	Email seen by the SDK — this is the field the identity map wants.
participant_uuid	STRING	SDK's participant ID.
inserted_at	TIMESTAMP	When the row was written.

3.4 How the three tables combine — field-by-field lineage

Where every column of presence_intervals actually comes from:

presence_intervals field	Comes from
interval_id	Generated fresh at build time (not from any raw table).
event_date	The day being built. By the login-date rule this is the IST day the session started , even if it ended after midnight (events from partition D+1 can land here).
meeting_id, meeting_uuid	participant_events_p.meeting_id (cast to STRING) / meeting_uuid.

<code>participant_key</code>	Email from the identity map when a cleaned name maps to exactly one email that day (pairs collected from both <code>participant_events_p</code> and <code>room_snapshots_v2</code>); otherwise the cleaned name itself.
<code>participant_name</code> , <code>participant_email</code>	<code>participant_events_p</code> (one representative value per person).
<code>room_name</code>	<code>participant_events_p.room_name</code> if it's a real name; if it's a <code>Room-xxxx</code> placeholder → the latest real name for that <code>room_uuid</code> , taken from either the event stream or <code>room_mappings</code> — whichever is newer.
<code>room_category</code>	Derived from the final room name: contains "break time" → <code>break</code> ; Main Room → <code>main</code> ; anything else → <code>breakout</code> .
<code>start_ts</code> , <code>end_ts</code>	The state machine's output over <code>participant_events_p.event_timestamp</code> values (join/leave pairs, split at each breakout move).
<code>duration_seconds</code>	<code>end_ts - start_ts</code> , computed at build time.
<code>alone_seconds</code> , <code>snapshot_count</code>	From bot snapshots on historical (pre-cutover) days; always 0 on webhook-built days — webhooks can't see who else is in the room.
<code>source</code> , <code>confidence</code>	Assigned by the builder based on how the row was derived (see the tables in section 5).
<code>built_at</code>	Timestamp of the build run that produced the row.

4. The Rules — every condition we apply, in plain words

Step A — Clean up the events

- **Name cleaning:** Zoom adds suffixes when people reconnect ("Kajal Yadav-1"). We strip these so all variants count as the same person.
- **Who is a person:** if a display name maps to exactly **one** email that day, we identify the person by email. If not, we fall back to the cleaned name.
- **The bot is excluded:** any name containing "scout" is dropped — it is our monitoring bot, not an employee.

Step B — Find each person's presence windows (when they were in the meeting)

Join–Leave pairing: a *join* event opens a window, a *leave* event closes it. One person can have many windows in a day (morning session, after-lunch session, etc.).

Room-move rule (5 seconds): when you move between breakout rooms, Zoom fires a fake "left the meeting" event. If a *leave* happens within 5 seconds of a breakout-room join **and** the person rejoins within 30 seconds, we treat it as a room move — not an exit. Without this rule people lost whole afternoons of hours.

Micro-disconnect rule (30 seconds): if someone drops and rejoins within 30 seconds (bad internet), the two windows are merged into one — no time lost, no fake break.

Login-date rule: a session belongs entirely to the IST day the person **logged in**. If you join at 22:00 and work until 02:00, all 4 hours count on the day you joined. (Same convention as Zoom's own report.)

08:00 IST guard: if a person has **no join event** and only room-move events before 08:00 IST, those events are treated as the tail of yesterday's meeting and ignored — this prevented "phantom hours" appearing on the wrong day.

Missing leave event: if someone's leave webhook never arrived, we close their window at the last leave-event time of the day, cap it at **4 hours**, and mark the row with low confidence (0.35) so it is visibly an estimate.

Step C — Split each window into rooms

- Everyone starts in the **Main Room**. Each *breakout_room_joined* event moves them to that room; each *breakout_room_left* moves them back to Main.
- **Double-send filter:** Zoom sometimes sends the same event twice — an identical event within 5 seconds is ignored.
- **Room names:** the name inside the webhook is the truth. Only when the webhook carries a placeholder code (`Room-xxxx`) do we look it up in `room_mappings`. If both exist, the newer information wins.
- **Room category:** each segment is tagged `main`, `breakout`, or `break` (any room whose name contains "Break Time").

Step D — Final safety caps and the hours total

- Segments shorter than **5 seconds** are dropped (webhook noise).
- No single segment can exceed **10 hours** (600 minutes).
- **Working hours = everything except "break" rooms.** Break Time is stored but not counted as work.
- **Minute rounding (Zoom convention):** every interval is billed as `CEILING(seconds / 60)` — a 61-second visit counts as 2 minutes, exactly like Zoom's own downloaded report. Totals are the **sum of rounded intervals**, never rounding the sum.

5. Output Table Schema — `presence_intervals`

Partitioned by `event_date`, clustered by `meeting_id`, `participant_key`. One row = one continuous stay of one person in one room.

Field	Type	What it means
<code>interval_id</code>	STRING (required)	Unique ID of this row (generated at build time).
<code>event_date</code>	DATE (required)	The IST working day this time belongs to (login-date rule). Partition key.
<code>meeting_id</code>	STRING	The Zoom meeting number the events came from.
<code>meeting_uuid</code>	STRING	Zoom's unique ID for that specific meeting instance.
<code>participant_key</code>	STRING (required)	Stable identity of the person: their email when it is unambiguous that day, otherwise the cleaned display name. All reports group by this.
<code>participant_name</code>	STRING	The display name as seen in Zoom (one representative variant).
<code>participant_email</code>	STRING	Email address if Zoom provided one.
<code>room_name</code>	STRING	The room the person was in (e.g. <code>0.Main Room</code> , <code>1.25: Victoria Memorial</code> , <code>BREAK TIME</code>).
<code>room_category</code>	STRING	<code>main</code> / <code>breakout</code> / <code>break</code> . Only <code>break</code> is excluded from working hours.
<code>start_ts</code>	TIMESTAMP (required)	When this room stay started (stored in UTC; add 5:30 for IST).
<code>end_ts</code>	TIMESTAMP (required)	When this room stay ended (UTC).
<code>duration_seconds</code>	INT64	<code>end_ts - start_ts</code> in seconds. Reports bill <code>CEILING(seconds/60)</code> minutes per row.
<code>alone_seconds</code>	INT64	Seconds the person was alone in the room (only known for old bot-snapshot days; 0 for webhook-built days).
<code>snapshot_count</code>	INT64	How many 30-second bot snapshots backed this row (old pipeline only; 0 for webhook-built days).
<code>source</code>	STRING	How the row was derived — see table below.
<code>confidence</code>	FLOAT64	How sure we are (0–1) — see table below. Days where most time is below 0.6 show a ⚠ "estimated" flag in the UI.
<code>built_at</code>	TIMESTAMP	When the builder produced this row (refreshes every ~2 minutes for today).

Values of `source`

source	Meaning
<code>webhook_room</code>	Breakout-room time built from webhook events (the normal case today).

webhook_fill	Main-Room time built from webhook events.
snapshot	Historical: directly observed by the bot's 30-second polling (pre 21 July 2026).
synthesized_main	Historical: Main-Room time inferred between bot snapshots.

Values of confidence

confidence	Meaning
1.0	Bot saw the person in the room (historical snapshot days).
0.6	Historical inferred Main-Room time.
0.5	Webhook-built row with a proper join and leave event (normal today).
0.35	Webhook-built row where the leave event was missing — end time estimated, capped at 4h.

6. The Query

Since 21 July 2026 the whole calculation runs **inside BigQuery** (no Python math). The full runnable script lives in the repo at `docs/build_presence_intervals_explained.sql`. Below is the same query with the JS state machine summarised — the SQL part is verbatim.

6.1 The state machine (JavaScript UDF — summarised)

```
CREATE TEMP FUNCTION build_intervals(events, monitoring_end, day_start, day_end, early_cutoff)
RETURNS ARRAY<STRUCT<room_name, room_category, source, confidence, start_ts, end_ts>>
LANGUAGE js AS """
    COALESCE_GAP_MS   = 30s    // merge disconnects shorter than this
    SEG_MIN_MS        = 5s     // drop segments shorter than this
    SEG_CAP_MS        = 600min // cap any single segment at 10h
    NO_LEAVE_CAP_MS    = 240min // cap at 4h when there is no leave webhook
    TRANSITION_GRACE   = 5s     // a 'left' within 5s of a breakout join = room move

    1. Sort the person's events by time.
    2. Pair joins with leaves into presence windows,
       ignoring 'left' events that are really room moves
       (left within 5s of a breakout join AND rejoin within 30s).
    3. Merge windows separated by a gap < 30s (micro-disconnects).
    4. Keep only windows that STARTED on this day (login-date rule).
       Fallback: if only breakout events exist and the first one is
       before 08:00 IST → it is yesterday's tail → return nothing.
    5. Tile each window into rooms using breakout_room_joined/left
       (5s duplicate filter; a stray 'left' with no join is ignored).
    6. Emit each room segment with source webhook_room / webhook_fill
       and confidence 0.5 (or 0.35 if the leave was missing).

    """;
```

6.2 The main SQL

```
DECLARE d          DATE          DEFAULT DATE '2026-07-22';
DECLARE day_start  TIMESTAMP DEFAULT TIMESTAMP_SUB(TIMESTAMP(d), INTERVAL 330 MINUTE); -- 00:00 IST

WITH
-- (1) The day's webhook events. Reads TWO partitions (d and d+1) so a session
--     crossing midnight is fully captured; only windows STARTING on d are kept.
base AS (
    SELECT
        norm_name(pe.participant_name)          AS name_key,
        pe.participant_name,
        pe.participant_email,
        CAST(pe.meeting_id AS STRING)           AS meeting_id,
        CAST(pe.event_timestamp AS TIMESTAMP) AS ts,
        pe.event_type                           AS et,
        -- Room name: newest knowledge wins (latest event name vs saved mapping)
        CASE
            WHEN pe.room_uuid IS NULL OR pe.room_uuid = '' THEN pe.room_name
            WHEN un.ev_name IS NOT NULL AND rm.mapped_name IS NOT NULL THEN
                IF(rm.mapped_at > un.ev_ts, rm.mapped_name, un.ev_name)
```



```

        WHEN un.ev_name IS NOT NULL THEN un.ev_name
        WHEN rm.mapped_name IS NOT NULL THEN rm.mapped_name
        ELSE pe.room_name
    END AS room
FROM `breakout_room_calibrator.participant_events_p` pe
LEFT JOIN ( -- latest SAVED name per room code
    SELECT room_uuid,
           ARRAY_AGG(room_name ORDER BY mapped_at DESC LIMIT 1)[OFFSET(0)] AS mapped_name,
           MAX(mapped_at) AS mapped_at
    FROM `breakout_room_calibrator.room_mappings`
    WHERE CAST(mapping_date AS STRING) = CAST(d AS STRING)
        AND room_name NOT LIKE 'Room-%'
    GROUP BY room_uuid
) rm ON pe.room_uuid = rm.room_uuid
LEFT JOIN ( -- latest name SEEN in the event stream per room code
    SELECT room_uuid,
           ARRAY_AGG(room_name ORDER BY event_timestamp DESC LIMIT 1)[OFFSET(0)] AS ev_name,
           MAX(event_timestamp) AS ev_ts
    FROM `breakout_room_calibrator.participant_events_p`
    WHERE event_date BETWEEN d AND DATE_ADD(d, INTERVAL 1 DAY)
        AND room_name NOT LIKE 'Room-%' AND room_name != 'Unknown Room'
        AND event_type IN ('breakout_room_joined', 'breakout_room_left')
    GROUP BY room_uuid
) un ON pe.room_uuid = un.room_uuid
WHERE pe.event_date BETWEEN d AND DATE_ADD(d, INTERVAL 1 DAY)
    AND pe.participant_name IS NOT NULL AND pe.participant_name != ''
    AND LOWER(pe.participant_name) NOT LIKE '%scout%' -- drop the bot
    AND pe.event_type IN ('participant_joined', 'participant_left',
                        'meeting.participant_joined', 'meeting.participant_left',
                        'breakout_room_joined', 'breakout_room_left')
),

-- (2) Identity: a name maps to an email only if EXACTLY ONE email that day.
ident_pairs AS (
    SELECT DISTINCT name_key, LOWER(TRIM(participant_email)) AS email
    FROM base WHERE participant_email IS NOT NULL AND TRIM(participant_email) != ''
    UNION DISTINCT
    SELECT DISTINCT norm_name(s.participant_name), LOWER(TRIM(s.participant_email))
    FROM `breakout_room_calibrator.room_snapshots_v2` s
    WHERE s.event_date = d AND s.participant_email IS NOT NULL
        AND LOWER(s.participant_name) NOT LIKE '%scout%'
),

ident AS (
    SELECT name_key, ANY_VALUE(email) AS mapped_email
    FROM ident_pairs GROUP BY name_key
    HAVING COUNT(DISTINCT email) = 1
),

-- (3) One row per person with their events bundled and sorted.
participants AS (
    SELECT
        COALESCE(i.mapped_email, b.name_key) AS participant_key,
        ANY_VALUE(b.participant_name) AS participant_name,
        ANY_VALUE(NULLIF(TRIM(COALESCE(b.participant_email, '')), '')) AS participant_email,
        ANY_VALUE(NULLIF(COALESCE(b.meeting_id, ''), '')) AS meeting_id,
        ARRAY_AGG(STRUCT(b.ts AS ts, b.et AS et, b.room AS room) ORDER BY b.ts) AS events
    FROM base b
    LEFT JOIN ident i ON b.name_key = i.name_key

```

```

GROUP BY participant_key
),

-- (4) monitoring_end = last leave webhook of the day
--      (closes windows that have no explicit leave).
mon AS (
  SELECT MAX(ts) AS monitoring_end
  FROM base WHERE et IN ('participant_left', 'meeting.participant_left')
)

-- (5) Run the state machine per person → room-wise interval rows.
SELECT
  p.participant_key, p.participant_name, p.participant_email, p.meeting_id,
  t.room_name, t.room_category,
  t.start_ts, t.end_ts,
  TIMESTAMP_DIFF(t.end_ts, t.start_ts, SECOND) AS duration_seconds,
  t.source, t.confidence
FROM participants p
CROSS JOIN mon
CROSS JOIN UNNEST(build_intervals(
  p.events, mon.monitoring_end, day_start,
  TIMESTAMP_ADD(day_start, INTERVAL 24 HOUR), -- day_end
  TIMESTAMP_ADD(day_start, INTERVAL 8 HOUR)   -- 08:00 IST early cutoff
)) AS t;

```

6.3 From rows to the number on the dashboard

```

SELECT participant_name,
  CAST(CEILING(SUM(IF(room_category != 'break', duration_seconds, 0))) / 60.0) AS INT64)
  AS working_mins
FROM presence_intervals
WHERE event_date = '2026-07-22'
GROUP BY participant_name, participant_key
ORDER BY working_mins DESC;

```

In the production UI each interval is first rounded up to whole minutes and then summed (Zoom's billing convention), so per-room minutes always add up to the person's total.

7. When the data refreshes

Job	Schedule (IST)	What it does
intervals-rebuild-today-2min	every 2 min, 08:00–23:59	Rebuilds today's partition (takes ~10–20 s).
intervals-rebuild-yesterday (06:00 & 12:30)	twice daily	Rebuilds yesterday after midnight-crossing sessions have ended.
intervals-auto-build-sweep	every 15 min	Self-heals any missing/outdated day in the last 35 days.
mapping-health-noon	12:00 daily	Emails an alert if no room mappings were saved today.

Good to know: our numbers reconcile with Zoom's own downloaded report row-for-row, with two accepted differences — (1) time before 08:00 IST with no join event is treated as the previous day's tail, and (2) people who hop rooms very fast can differ by a few minutes because each hop's ~3-second gap is rounded on both sides.